

Detección y mitigación de vulnerabilidades Cross-Site Scripting (XSS) en aplicaciones web desarrolladas con Laravel

Detection and Mitigation of Cross-Site Scripting (XSS) Vulnerabilities in Laravel-Based Web Applications

Cruz Bryan¹, Erazo Stefany¹, Durán Giovanni¹

¹ Universidad de las Fuerzas Armadas ESPE Sede Santo Domingo, Ecuador.
bscruz@espe.edu.ec, scerazo1@espe.edu.ec, gfduran@espe.edu.ec

Resumen

Las aplicaciones web constituyen un componente crítico de la infraestructura digital contemporánea; sin embargo, su exposición constante y el procesamiento de entradas controladas por usuarios incrementan la probabilidad de vulnerabilidades como Cross-Site Scripting (XSS). Este estudio evalúa la detección y mitigación de XSS almacenado en una aplicación desarrollada con el framework Laravel, mediante una metodología experimental ejecutada en un entorno controlado. La investigación combinó análisis estático y dinámico: por un lado, se realizó revisión del código fuente para identificar decisiones de renderizado en plantillas Blade con potencial impacto en la seguridad (p. ej., salida no escapada); por otro, se efectuaron pruebas activas orientadas a reproducir condiciones de explotación. Los resultados mostraron que los intentos de inyección desde la interfaz cliente presentaron limitaciones debido a mecanismos de saneamiento del editor de texto enriquecido; no obstante, la vulnerabilidad se confirmó en el backend al comprobarse que cargas maliciosas podían enviarse directamente a los endpoints de persistencia mediante solicitudes HTTP controladas, manteniendo el contexto de sesión a través de controles de autenticación y protección CSRF. Como evidencia del impacto, el contenido inyectado fue almacenado en la base de datos y, al ser recuperado y renderizado posteriormente sin un tratamiento adecuado, ejecutó código JavaScript en el navegador, validando un escenario de Stored XSS con alcance potencial sobre múltiples usuarios. En respuesta, se implementó una estrategia de mitigación centrada en el servidor basada en sanitización por lista blanca, utilizando una librería especializada para restringir etiquetas, atributos y propiedades permitidas a un subconjunto compatible con requisitos de formato, y eliminando elementos asociados a contenido activo. Tras la mitigación, las pruebas evidenciaron que las cargas peligrosas dejaron de persistirse en su forma original y no se observó ejecución de scripts, preservando la funcionalidad prevista.

Palabras clave: Vulnerabilidades, detección, mitigación, XSS, Laravel.

Abstract

Web applications constitute a critical component of contemporary digital infrastructure; however, their continuous exposure and the processing of user-controlled input increase the likelihood of vulnerabilities such as Cross-Site Scripting (XSS). This study evaluated the detection and mitigation of stored XSS in an application developed with the Laravel framework through an experimental methodology conducted in a controlled environment. The research combined static and dynamic analysis: first, the source code was reviewed to identify rendering

decisions in Blade templates with potential security impact (e.g., unescaped output); second, active tests were performed to reproduce exploitation conditions. The results showed that injection attempts from the client interface were constrained by sanitization mechanisms in the rich-text editor; nevertheless, the vulnerability was confirmed on the backend by demonstrating that malicious payloads could be sent directly to persistence endpoints via controlled HTTP requests, while preserving session context through authentication controls and CSRF protection. As evidence of impact, the injected content was stored in the database and, when later retrieved and rendered without adequate handling, executed JavaScript code in the browser, validating a stored XSS scenario with potential impact on multiple users. In response, a server-side mitigation strategy based on whitelist sanitization was implemented using a specialized library to restrict allowed tags, attributes, and properties to a subset compatible with formatting requirements, while removing elements associated with active content. After mitigation, testing confirmed that dangerous payloads were no longer persisted in their original form and no script execution was observed, while the intended functionality was preserved.

Keywords: Vulnerabilities, detection, mitigation, XSS

I. Introducción

La seguridad de las aplicaciones web esta condicionada por fallas de inyección en la capa de presentación, entre ellas Cross-Site Scripting (XSS) destaca por su impacto, dado que este permite que un atacante inyectar scripts en supuestas páginas “confiables” y ejecute código en el navegador de la víctima cuando la aplicación incorpora entrada no confiable en la salida sin el tratamiento adecuado (OWASP Foundation, 2026). Estudios comparativos recientes sobre frameworks muestran que, aunque ya existan mecanismos de protección, la resistencia real depende de cómo se aplican las reglas ante técnicas como XSS y CSRF (Ramos, 2022).

En Laravel, la superficie de ataque suele crecer en funcionalidades, generalmente, módulos que admiten texto enriquecido (WYSIWYG), por todo lo mencionado OWASP recomienda que, cuando el usuario necesita escribir HTML, se aplique la sanitización de HTML, y no sólo confiar en filtros ad-hoc (*OWASP Cheat Sheet Series*, 2026).

Además, las vulnerabilidades públicas reportadas en editores de texto enriquecido integrados en aplicaciones evidenciaron que el contenido enriquecido constituyó un vector recurrente para XSS almacenado cuando no se sanitizó de manera adecuada.

El propósito de este análisis es detectar y mitigar vulnerabilidades XSS que se presentan en aplicaciones desarrolladas con Laravel, describiendo un proceso que conecte el problema con controles concretos conocidos del framework. En particular, se analiza cómo tomar decisiones de renderizado en Blade (p. ej., uso de `{{ }}` vs. `{!! !!}`), helpers de escape, y el manejo de entradas con HTML afectan el riesgo de XSS. Laravel documenta que `{{ }}` aplica escape para reducir XSS, mientras que la salida no escapada con `{!! !!}` debe usarse con extrema precaución (Laravel, 2026).

Finalmente, los hallazgos que se reportan se centran en que la mayor probabilidad de XSS aparece cuando la aplicación renderiza contenido suministrado por usuarios como texto no escapado, o cuando se permite texto enriquecido sin una sanitización robusta; en ambos casos, el XSS puede degradar la confianza de otras defensas del lado servidor al ejecutar acciones con el contexto de la sesión de la víctima (*OWASP Foundation*, 2026).

El artículo se organizó de la siguiente manera: en Materiales y Métodos se describió el entorno experimental, las herramientas empleadas y el procedimiento aplicado para la detección y mitigación; en Evaluación de Resultados y Discusión se presentaron los

hallazgos antes y después de la mitigación, junto con su análisis; finalmente, en Conclusiones se sintetizaron los aportes y se plantearon consideraciones finales

II. Materiales y Métodos

II.1 Entorno experimental y herramientas

El estudio se ejecutó en un entorno controlado local con sistema operativo Windows 11, la aplicación web se desarrolló y desplegó usando Laravel 10, iniciada mediante el servidor de desarrollo (php artisan serve). La base de datos relacional fue MySQL versión 8.4, administrada localmente con Laragon.

Para la interacción y evaluación de solicitudes HTTP se empleó Postman, con el fin de observar y reproducir el comportamiento del backend bajo diferentes entradas y flujos de autenticación. La aplicación incluyó un módulo de autenticación (login) y una funcionalidad de registro de “novedades” mediante un formulario con editor de texto enriquecido.

II.2 Diseño del caso de uso: entrada con texto enriquecido en Blade

Con base en la documentación y prácticas comunes en Laravel, se identificó que las vistas Blade renderizan variables escapadas usando `{{ ... }}`, lo cual mitiga la ejecución de HTML/JavaScript inyectado. No obstante, en escenarios donde existen requerimientos funcionales para permitir texto enriquecido (por ejemplo, negritas, subrayado, listas o enlaces, típico en foros o sistemas documentales), se recurre a la impresión sin escape mediante `{!! ... !!}`, habilitando la representación de etiquetas HTML en la vista.

Por lo tanto, se implementó una funcionalidad que permitía al usuario ingresar contenido con formato en un campo tipo textarea, incorporando un editor WYSIWYG (Summernote) para restringir la barra de herramientas a estilos básicos (negrita, subrayado, listas, enlaces) e incluir una opción de vista de código. La finalidad fue simular un requisito realista de negocio que, de no controlarse adecuadamente, incrementa el riesgo de Cross-Site Scripting (XSS) cuando el contenido se almacena y luego se presenta a otros usuarios.

II.3 Identificación y comprobación del vector de inyección

En una primera fase, se verificó si el vector de inyección era viable desde el frontend. Como se ve en la Figura 1 al introducir contenido con intención de ejecutar JavaScript desde el editor, se observó que la librería del editor aplicaba mecanismos de saneamiento del texto ingresado, impidiendo que cargas simples de scripts se ejecuten desde la interfaz, a pesar que en el archivo en blade ya está configurado con la impresión sin escape `"{!! $item->novelty !!}"`. Con ello, se determinó que la superficie de ataque no era efectiva a través del cliente en ese punto.

The image shows a web application interface. At the top is a form titled "NUEVA NOVEDAD" with a close button. Below the title is a label "Detalle la novedad (permite negrita y subrayado):". The form has a rich text editor toolbar with buttons for bold (B), underline (U), bulleted list, numbered list, link, and source code (</>). The text area contains the payload: `<script>alert("prueba de ataque xss")</script>`. Below the text area is a hint: "Puedes usar formato: **negrita**, subrayado, listas y enlaces." At the bottom right of the form are two buttons: "CANCELAR" (red) and "GUARDAR" (green).

Below the form is a table with a header "NOVEDADES". The table has two columns: "ORD" and "NOVEDADES". The first row shows the number "1" in the "ORD" column and the same XSS payload in the "NOVEDADES" column. A green icon is visible at the end of the row.

Figura 1: “Intento de ataque XSS desde frontend”

Por ese motivo el análisis se enfocó en el flujo del backend, evaluando directamente los endpoints involucrados en autenticación y almacenamiento del contenido de las novedades. Para ello se reprodujo desde postman el flujo de sesión de un usuario legítimo, considerando que un usuario cuenta con un email y una contraseña válidos, contemplando:

Acceso a recursos del sistema mediante un método GET que se encuentre expuesto. Con el fin de obtener los valores de seguridad expuestos en la respuesta HTML, específicamente el valor del token CSRF que viene en la respuesta de postman. La Figura 2 muestra como se tiene acceso al mencionado token desde postman.

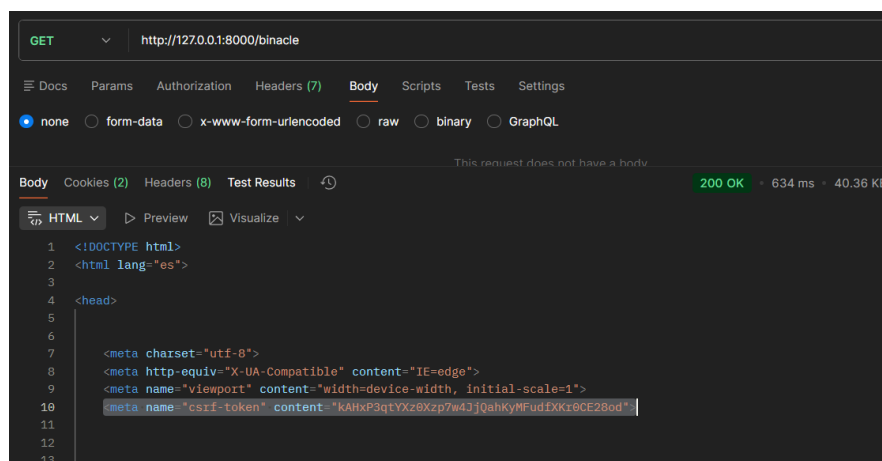


Figura 2: “Captura de token CSRF”

Ejecución del inicio de sesión mediante solicitud POST con credenciales válidas y con el token CSRF como parámetro de cabecera se envió desde postman la solicitud al endpoint post: <http://127.0.0.1:8000/login>. La finalidad es obtener la cookie de sesión que viene como respuesta en la sección de cookies. En la Figura 3 se muestra como se capturó la mencionada cookie.

Key	Value	Description
Accept	text/html	
Content-Type	application/json	
X-Requested-With	XMLHttpRequest	
X-CSRF-TOKEN	wgafaDv0y8lgFfmqMY5toAt5nwMqLdszABXbGmj3	

Name	Value	Domain	Path	Expires
XSRF-TOKEN	eyJpdll6ljhRQ1p4QXh...	127.0.0.1	/	Thu, 29 Jan 2026 17:4...
laravel2_session	eyJpdll6lRzT1NxOWJ...	127.0.0.1	/	Thu, 29 Jan 2026 17:4...

Figura 3: “Captura de cookie de sesión”

Invocación del endpoint de creación/almacenamiento con el token CSRF y con el cookie de sesión se tuvieron los parámetros necesarios para enviar una solicitud al endpoint post: <http://127.0.0.1:8000/binnacle/newNovelty> con el script de ataque XSS. En primer lugar se hizo una pequeña prueba enviando un script que muestre un mensaje en consola, como se puede ver en la Figura 4 y su ejecución en la Figura 5.

```
{
  "novelty": "<script>console.log('Prueba de acceso a ataques')</script>"
}
```


Name	Value	Domain	Path	Expires	HttpOnly
XSRF-TOKEN	eyJpdll6lm5XMYyNT...	127.0.0.1	/	Thu, 29 Jan 2026 20:...	false
laravel2_session	eyJpdll6lnUzaTdPamp...	127.0.0.1	/	Thu, 29 Jan 2026 20:...	true

Figura 4: “Envío de script de prueba”

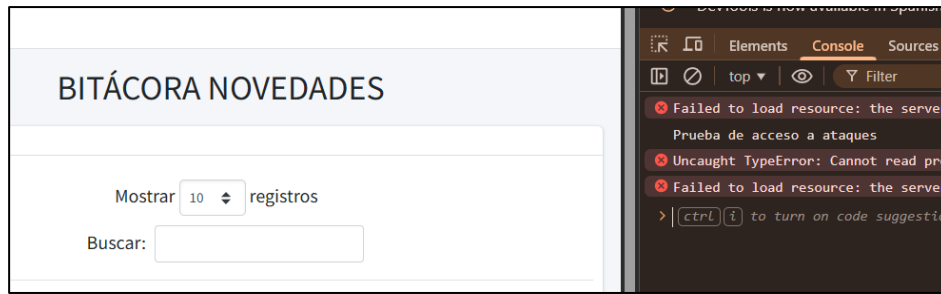


Figura 5: “Mensaje en consola con texto enviado mediante script”

Una vez confirmado que es posible el ataque, se envía un Script “malicioso” como se ve en la Figura 6.



Figura 6: “Script malicioso enviado por postman”

Como evidencia de impacto, se verificó que el contenido enviado por backend se almacenó en la base de datos y, al ser renderizado posteriormente en la vista, provocó la ejecución de JavaScript en el navegador, confirmando un escenario de XSS almacenado (Stored XSS). Ver Figura 7.

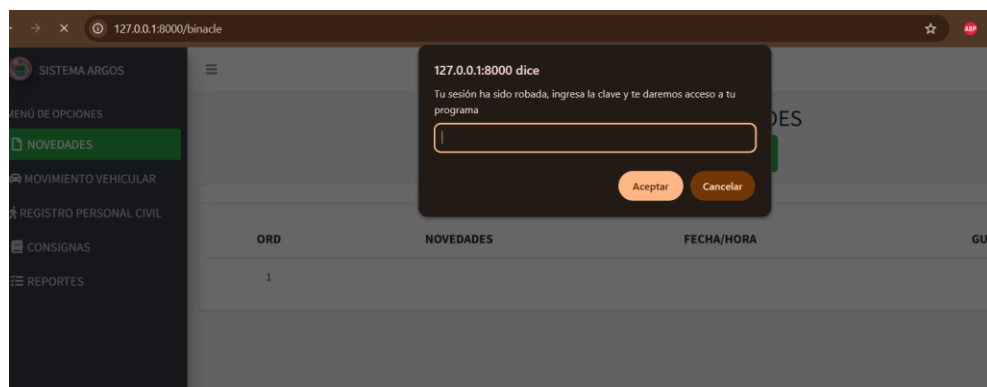


Figura 7: “Resultado de página vulnerada”

Es importante mencionar que el script malicioso se ejecutará en todas las máquinas y/o usuarios que accedan a esa aplicación, ya que es un script que se preserva en la base de datos general, no en la sesión del usuario ni en la máquina.

Aclaración metodológica relevante: debido a que la aplicación fue desarrollada por los autores de este artículo, se contaba con conocimiento previo de las rutas, métodos y parámetros utilizados por el sistema. En un escenario real, un atacante debería realizar un proceso adicional de **enumeración y descubrimiento** (p. ej., revisión de rutas expuestas, análisis de formularios, inspección de tráfico, pruebas de endpoints) para identificar endpoints y nombres de parámetros antes de intentar una inyección. En este trabajo, dicho paso se

omitió por tratarse de una validación controlada y reproducible del comportamiento del backend.

II.4 Propuesta de Mitigación del XSS almacenado mediante sanitización por lista blanca

Una vez confirmada la vulnerabilidad, se implementó una estrategia de mitigación centrada en el servidor, considerando que los controles del lado del cliente (p. ej., restricciones del editor WYSIWYG) no constituyen una defensa suficiente. La mitigación se aplicó en el controlador responsable del almacenamiento del contenido, antes de persistir los datos en base de datos.

La Figura 8 muestra el código implementando en el controlador que permite ejecutar la mitigación.

```
use Mews\Purifier\Facades\Purifier;

class NoveltyController extends Controller
{
    // Método para sanitizar el HTML y mitigar el riesgo
    private function sanitizeNovelty(string $original): string
    {
        $cleanHtml = Purifier::clean($original, [
            'HTML.Allowed' => 'b,strong,i,em,u,s,span,p,br,ul,ol,li,blockquote',
            'CSS.AllowedProperties' => 'font,font-size,font-weight,font-style,text-decoration,text-align,color,background-color',
            'HTML.ForbiddenElements' => 'script,iframe,object,embed',
        ]);

        if (trim($original) !== '' && trim(strip_tags($cleanHtml)) === '') {
            return '<p><b>Intento de ataque XSS detectado</b></p>';
        }

        return $cleanHtml;
    }
}
```

Figura 8: “Código para limpieza de input de html”

Se incorporó la librería Mews Purifier, utilizada para la sanitización del contenido HTML mediante reglas configurables.

El objetivo fue permitir únicamente un subconjunto de etiquetas y estilos requeridos por el caso de uso (texto enriquecido básico), eliminando elementos asociados a ejecución de scripts o contenidos activos.

Este enfoque reduce el riesgo de XSS almacenado al impedir que cargas con etiquetas o atributos peligrosos queden almacenadas en la base de datos, lo cual también disminuye la superficie de impacto si el contenido es mostrado posteriormente usando renderizado de HTML

Política de lista blanca (allowlist) y bloqueo de elementos peligrosos

El método aplicado para la sanitización fue configurado con políticas basadas en:

- Lista blanca de etiquetas HTML permitidas, enfocada en formato y estructura de texto (negrita, cursiva, subrayado, listas, párrafos y saltos de línea).
- Lista blanca de propiedades CSS permitidas, limitando estilos a propiedades relacionadas con formato tipográfico y presentación básica.

- Lista negra explícita de elementos peligrosos, denegando etiquetas típicamente asociadas a ejecución de contenido activo como `'script,iframe,object,embed'`.

De esta manera, cualquier intento de incluir JavaScript embebido, contenido activo o mecanismos típicos de ejecución (por ejemplo, etiquetas `<script>` o inclusión de objetos embebidos) fue eliminado del contenido antes de su persistencia.

Adicional, se incorporó un método para registrar en la base de datos un intento de ataque XSS. Lo que permite ver en la base de datos el listado del usuario y el intento de ataque XSS que pretendía ejecutar.

III. Evaluación de Resultados y Discusión

III.1 Evaluación de Resultados (código con mitigación)

Los resultados obtenidos durante el desarrollo del entorno experimental permitieron confirmar la presencia de una vulnerabilidad de Cross-Site Scripting (XSS) almacenado cuando el contenido ingresado por el usuario fue renderizado en vistas Blade sin aplicar mecanismos adecuados de control en la salida. En la fase inicial, se observó que los intentos de inyección realizados desde el frontend no generaron un impacto directo debido a los mecanismos de saneamiento implementados por el editor WYSIWYG, lo cual redujo la probabilidad de ejecución de scripts simples desde la interfaz gráfica.

Como se muestra en la Figura 9, en un ataque XSS almacenado el código malicioso es persistido en la base de datos de la aplicación y, posteriormente, ejecutado en el navegador de la víctima cuando el contenido vulnerable es renderizado..

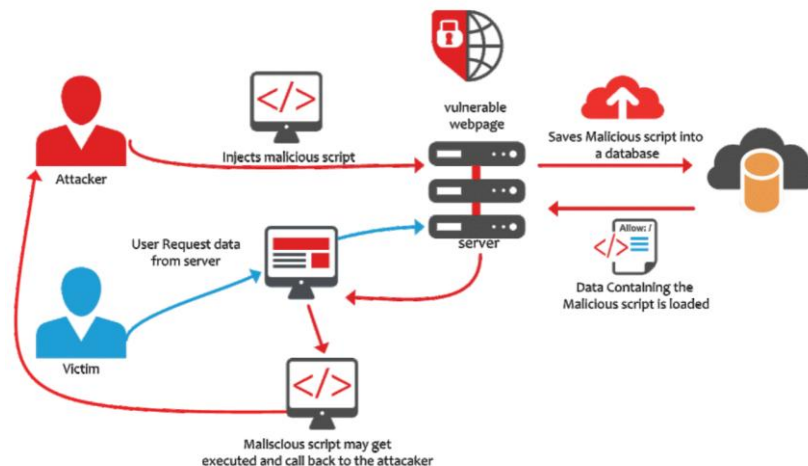


Figura 9: “Esquema general de un ataque Cross-Site Scripting (XSS) almacenado”

No obstante, al evaluar el flujo del backend mediante solicitudes controladas, se comprobó que el sistema permitió el almacenamiento de contenido con código potencialmente peligroso cuando este fue enviado directamente a los endpoints responsables de la persistencia de datos. Una vez almacenado, dicho contenido fue recuperado y presentado en la vista correspondiente, lo que provocó la ejecución de código en el navegador del usuario que accedió posteriormente

a la funcionalidad afectada. Este comportamiento corroboró un escenario de XSS almacenado, dado que el script permaneció persistido en la base de datos y se ejecutó de forma automática cada vez que la página fue cargada.

Posteriormente, tras la implementación de la estrategia de mitigación basada en sanitización por lista blanca en el lado del servidor, se evidenció un cambio significativo en el comportamiento de la aplicación. Los intentos de inyección realizados después de aplicar la sanitización no fueron persistidos en la base de datos en su forma original, debido a que los elementos asociados a la ejecución de código fueron eliminados antes del almacenamiento. Como resultado, el contenido mostrado en la vista mantuvo el formato permitido por el caso de uso, pero sin ejecutar scripts ni generar efectos colaterales en el navegador del usuario.

La Figura 2 representa el flujo general de mitigación de ataques XSS, el cual inició con la identificación de entradas controladas por el usuario y continuó con la detección de posibles vulnerabilidades mediante análisis estático y dinámico

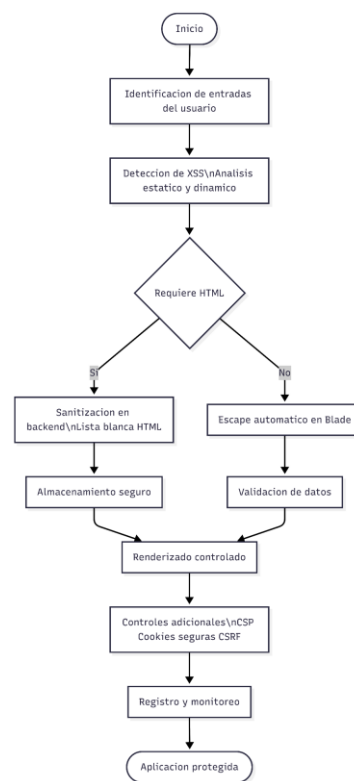


Figura 10: “Flujograma simplificado del proceso de mitigación de vulnerabilidades Cross-Site Scripting (XSS) en aplicaciones web desarrolladas con Laravel”

Adicionalmente, la incorporación de mecanismos de registro permitió identificar los intentos de ataque y asociarlos con el usuario y el contenido enviado, lo que aportó evidencia adicional sobre la efectividad de la mitigación aplicada. En conjunto, los resultados evidenciaron una reducción significativa del riesgo de explotación de XSS almacenado, lo que validó la eficacia de la estrategia implementada en el backend.

En la Tabla 1 se presenta una comparación del comportamiento de la aplicación antes y después de la implementación de las medidas de mitigación. Como se observa, en la versión inicial se permitió la persistencia y ejecución de scripts maliciosos en un escenario de XSS almacenado.

Tabla 1: “Tabla comparativa del comportamiento de la mitigación”

Aspecto evaluado	Antes de la mitigación	Después de la mitigación
Control de entrada del usuario	Saneamiento parcial en el frontend por el editor WYSIWYG (no suficiente como control de seguridad)	Sanitización allowlist aplicada en el backend antes de la persistencia (con reglas configurables)
Persistencia de scripts maliciosos	El contenido potencialmente peligroso podría almacenarse en la base de datos	Los elementos peligrosos fueron eliminados antes del almacenamiento
Renderizado en vistas Blade	Salida HTML sin escape en Blade (p. ej., {!! ... !!}), sin sanitización en servidor	Salida HTML sin escape en Blade, pero con contenido previamente sanitizado en servidor (reducción del riesgo de ejecución)
Ejecución de JavaScript inyectado	Se confirmó la ejecución de scripts al cargar la vista	No se observó ejecución de código malicioso
Impacto sobre otros usuarios	El contenido malicioso afectaba a cualquier usuario que accedía a la vista	El contenido mostrado no generó efectos no deseados
Registro de intentos de ataque	No se implementó un mecanismo específico de registro de intentos XSS	Se registraron intentos de inyección (usuario + payload) en base de datos para análisis posterior.
Nivel de riesgo XSS	Bajo en frontend. Alto en backend, debido a la persistencia del código inyectado	Bajo en frontend y backend, tras aplicar sanitización y controles adicionales

III.2 Discusión

Los resultados obtenidos en este estudio permiten interpretar que la presencia de mecanismos de protección nativos en frameworks modernos como Laravel no garantiza, por sí sola, la ausencia de vulnerabilidades XSS, sino que su efectividad depende directamente de cómo dichos mecanismos son utilizados durante el desarrollo. En particular, se observó que la decisión de renderizar contenido sin escape en las vistas Blade, aun cuando responde a requerimientos funcionales legítimos, incrementa de manera considerable la superficie de ataque si no se acompaña de controles adicionales en el servidor.

En comparación con lo reportado en la literatura, los hallazgos concuerdan con las recomendaciones de OWASP y con estudios previos que señalan que el XSS almacenado suele originarse cuando el contenido generado por el usuario es persistido y posteriormente presentado sin un proceso robusto de sanitización. Tal como se ha documentado en investigaciones similares, los controles implementados únicamente en el frontend resultaron insuficientes frente a un atacante que interactúa directamente con los endpoints del backend, lo que refuerza la necesidad de aplicar validaciones y sanitización del lado del servidor.

Un aspecto relevante identificado fue que la mitigación basada en listas blancas permitió conservar la funcionalidad requerida por el sistema, al mismo tiempo que eliminó elementos peligrosos asociados a la ejecución de código. Este resultado confirma que es posible equilibrar los requisitos de usabilidad y seguridad cuando se emplean estrategias de sanitización adecuadas, alineadas con las buenas prácticas de desarrollo seguro. Asimismo, la integración de controles complementarios, como el uso de políticas de seguridad y cookies con atributos restrictivos, contribuyó a reforzar la defensa en profundidad de la aplicación.

No obstante, el estudio presentó ciertas limitaciones que deben ser consideradas. El entorno experimental fue controlado y desarrollado por los propios autores, lo que permitió un conocimiento previo de las rutas y parámetros del sistema, situación que difiere de un escenario real donde un atacante debería realizar procesos adicionales de descubrimiento. Además, el análisis se centró exclusivamente en vulnerabilidades XSS, sin evaluar su posible encadenamiento con otros vectores de ataque, lo cual representa una línea de investigación futura.

En términos generales, los resultados y su análisis permiten concluir que las vulnerabilidades XSS en aplicaciones Laravel no responden a fallas inherentes del framework, sino a prácticas de desarrollo inseguras o a un uso inadecuado de las herramientas disponibles. El estudio demuestra que la combinación de detección temprana, sanitización del lado del servidor y controles de seguridad complementarios constituye una estrategia efectiva para mitigar este tipo de ataques en aplicaciones web modernas.

IV. Conclusiones y Trabajo futuro

El análisis realizado permitió detectar y confirmar la existencia de una vulnerabilidad de tipo Cross-Site Scripting almacenado en una aplicación web desarrollada con Laravel, evidenciando que el uso incorrecto de renderizado no escapado representa un riesgo significativo para la seguridad, adicional se demostró que la implementación de sanitización basada en listas blancas constituye una medida adecuada y efectiva para mitigar este tipo de ataques sin afectar la funcionalidad requerida por el sistema.

Los resultados obtenidos resaltan la importancia de integrar prácticas de desarrollo seguro, combinando el uso adecuado de los mecanismos nativos del framework con herramientas especializadas de seguridad.

Como trabajo futuro propuesto, sería extender la investigación hacia la identificación y mitigación de vulnerabilidades de tipo Cross-Site Scripting en aplicaciones web desarrolladas con diferentes frameworks como Spring Boot y .NET, ya que se tiene conocimiento que son tecnologías ampliamente utilizadas en sistemas informáticos de instituciones públicas, el objetivo a alcanzar será analizar cómo estos entornos gestionan el tratamiento de entradas de usuario y el renderizado de contenido dinámico, así como evaluar la efectividad de sus mecanismos de seguridad nativos frente a escenarios de XSS almacenado y reflejado, con el fin de fortalecer los lineamientos de desarrollo seguro en aplicaciones web institucionales.

Referencias Bibliográficas

Cross Site Scripting (XSS) | OWASP Foundation. (2026). Owasp.org.

https://owasp.org/www-community/attacks/xss/?utm_source=chatgpt.com

Ramos, E. (2022). Análisis de seguridad de XSS, SQL Injection y CSRF en Laravel,

Django, Express y Spring. Eafit.edu.co; Universidad EAFIT.

<https://repository.eafit.edu.co/entities/publication/b8efc186-a4a1-4a4f-b1e5-c8e2155caab0>

Cross Site Scripting Prevention - OWASP Cheat Sheet Series. (2026). Owasp.org.

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html?utm_source

Laravel. (2026). Blade Templates - Laravel 12.x - The PHP Framework For Web Artisans.

Laravel.com. https://laravel.com/docs/12.x/blade?utm_source=chatgpt.com#custom-if-statements

Eriksson, B., Picazo-Sanchez, P., & Sabelfeld, A. (2022). Hardening the security analysis of browser extensions. *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*, 1694–1703. <https://doi.org/10.1145/3477314.3507098>

Ramos, E. (2022). Análisis de seguridad de XSS, SQL Injection y CSRF en Laravel,

Django, Express y Spring. Eafit.edu.co; Universidad EAFIT.

<https://repository.eafit.edu.co/entities/publication/b8efc186-a4a1-4a4f-b1e5-c8e2155caab0>

Ibarra. (2019, October). *Desarrollo de un plan de auditorías de seguridad a sistemas web basados en el framework laravel utilizando las plataformas Windows y Linux*. Buap.mx. <https://repositorioinstitucional.buap.mx/items/9c0e39be-ef54-4e70-80d5-08b8d57d0064>